

Range Update Queries

Given an array of n integers, your task is to process q queries of the following types:

1. increase each value in range $[a,b]$ by u
2. what is the value at position k ?

Input

The first input line has two integers n and q : the number of values and queries.

The second line has n integers $x_1, x_2, \dots, x_n$: the array values.

Finally, there are q lines describing the queries.
Each line has three integers: either "1 a b u " or "2 k ".

Output

Print the result of each query of type 2.

Constraints

$1 \leq n, q \leq 2 \cdot 10^5$
 $1 \leq x_i, u \leq 10^9$

$1 \leq k \leq n$
 $1 \leq a \leq b \leq n$

Example

Input:
8 3
3 2 4 5 1 1 5 3
2 4
1 2 5 1
2 4

Output:
5
6

Why a Normal Fenwick Tree Doesn't Work Directly

A standard Fenwick Tree supports:

- Point update
- Prefix query

But here we need:

- Range update
- Point query

At first glance, this feels opposite.

Instead of changing the data structure, we change what the Fenwick Tree represents.

Core Idea (Difference Array Thinking)

Let's forget Fenwick Tree for a moment.

Define a difference array D :

```
D[1] = A[1]
D[i] = A[i] - A[i-1] for i > 1
```

Then:

```
A[k] = D[1] + D[2] + ... + D[k]
```

So the value at position k is just a prefix sum of D .

What Happens to Difference Array on Range Update?

Suppose we do:

increase $A[a..b]$ by u

Effect on D :

```
D[a] += u
D[b+1] -= u (if b+1 ≤ n)
```

Everything else stays the same.

This is a huge simplification:

- Range update becomes two point updates
- Point query becomes prefix sum

Algorithm Using Fenwick Tree

```
Initialization
    Build Fenwick Tree over difference array D

Query Type 1: 1 a b u
    - update(a, +u)
    - update(b + 1, -u) // if b+1 ≤ n

Query Type 2: 2 k
    answer = query(k) // prefix sum
```

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<long long> bit;

    Fenwick(int n) : n(n), bit(n + 1, 0) {}

    void update(int idx, long long val) {
        while (idx <= n) {
            bit[idx] += val;
            idx += idx & -idx;
        }
    }

    long long query(int idx) {
        long long sum = 0;
        while (idx > 0) {
            sum += bit[idx];
            idx -= idx & -idx;
        }
        return sum;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    Fenwick fw(n);

    vector<long long> A(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> A[i];
    }

    // Build difference array in Fenwick Tree
    fw.update(1, A[1]);
    for (int i = 2; i <= n; i++) {
        fw.update(i, A[i] - A[i - 1]);
    }

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) {
            int a, b;
            long long u;
            cin >> a >> b >> u;
            fw.update(a, u);
            if (b + 1 <= n) {
                fw.update(b + 1, -u);
            }
        } else {
            int k;
            cin >> k;
            cout << fw.query(k) << "\n";
        }
    }
}
```